

Week 8 Skill

TO Parse Combined Redirection, Duplicate File Descriptors, Merge Streams, Verify Ordering, Handle Edge Cases, Test Integrated Outputs

```
divya@Divyasree:~$ cd
divya@Divyasree:~$ mkdir combined_redirect_project
divya@Divyasree:~$ cd combined_redirect_project
divya@Divyasree:~/combined_redirect_project$ pwd
/home/divya/combined_redirect_project
divya@Divyasree:~/combined_redirect_project$ nano combined_redirect.c
divya@Divyasree:~/combined_redirect_project$ gcc combined_redirect.c -o combined_redirect
divya@Divyasree:~/combined_redirect_project$ ls
combined_redirect  combined_redirect.c
divya@Divyasree:~/combined_redirect_project$ ./combined_redirect
===== Combined Redirection Manager =====

Supported syntax:
command > output.txt 2>&1

Example:
sh -c 'echo OUTPUT; echo ERROR >&2' > result.txt 2>&1

Enter command:
> sh -c 'echo OUTPUT; echo ERROR >&2' > result.txt 2>&1

Parsed Command: sh -c 'echo OUTPUT; echo ERROR
Output File: &2' > result.txt

Executing combined redirection...
Output file opened: &2' > result.txt
File descriptor: 3
Child process completed.
Child exit status: 2

===== Combined Redirection Test Completed =====
divya@Divyasree:~/combined_redirect_project$ cat result.txt
cat: result.txt: No such file or directory
```

divya@Divyasree: ~/combine X + v

```
Supported syntax:
command > output.txt 2>&1

Example:
sh -c 'echo OUTPUT; echo ERROR >&2' > result.txt 2>&1

Enter command:
> echo OUTPUT > result.txt 2>&1

Parsed Command: echo OUTPUT
Output File: result.txt

Executing combined redirection...
Output file opened: result.txt
File descriptor: 3
Child process completed.
Child exit status: 0

===== Combined Redirection Test Completed =====
divya@Divyasree:~/combined_redirect_project$ cat result.txt
OUTPUT
divya@Divyasree:~/combined_redirect_project$ nano stream_test.c
divya@Divyasree:~/combined_redirect_project$ gcc stream_test.c -o stream_test
divya@Divyasree:~/combined_redirect_project$ ls
'&2\' ' > result.txt' combined_redirect.c  stream_test
combined_redirect result.txt stream_test.c
divya@Divyasree:~/combined_redirect_project$ ./combined_redirect
===== Combined Redirection Manager =====

Supported syntax:
command > output.txt 2>&1
```

```

command > output.txt 2>&1

Example:
sh -c 'echo OUTPUT; echo ERROR >&2' > result.txt 2>&1

Enter command:
> ./stream_test > combined.txt 2>&1

Parsed Command: ./stream_test
Output File: combined.txt

Executing combined redirection...
Output file opened: combined.txt
File descriptor: 3
Child process completed.
Child exit status: 0

==== Combined Redirection Test Completed ====
divya@Divyasree:~/combined_redirect_project$ cat combined.txt
This is standard error.
This is standard output.
divya@Divyasree:~/combined_redirect_project$ |

```

Code :

```

GNU nano 7.2 combined_redirect.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_INPUT 300
#define MAX_ARGS 30

/*
Supported syntax:

command > file 2>&1

Example:

sh -c 'echo OUTPUT; echo ERROR >&2' > result.txt 2>&1
*/

/* Remove leading and trailing spaces */
void trim(char *str)
{
    char *start = str;

    while (*start == ' ')
        start++;

    if (start != str)

```

```

    int len = strlen(str);

    while (len > 0 && str[len - 1] == ' ')
    {
        str[len - 1] = '\0';
        len--;
    }
}

```

```

/*
    Parse combined redirection.

```

```

    Finds:

```

```

        >

```

```

    and checks for:

```

```

        2>&1

```

```

*/
int parse_combined(char *input,
                  char *command,
                  char *filename)
{
    char *redirect = strstr(input, ">");

    if (redirect == NULL)
    {
        return 0;
    }
}

```

```

    if (redirect == NULL)
    {
        return 0;
    }

    /*
        Make sure this is not >>.
    */

    if (*(redirect + 1) == '>')
    {
        printf("Error: Append redirection is not supported here.\n");
        return 0;
    }

    /*
        Check for 2>&1.
    */

    char *merge = strstr(redirect, "2>&1");

    if (merge == NULL)
    {
        printf("Error: Expected combined redirection '2>&1'.\n");
        return 0;
    }

    /*
        Separate command from redirection.

```

```
GNU nano 7.2                                     combined_redirect.c
*/

char *merge = strstr(redirect, "2>&1");

if (merge == NULL)
{
    printf("Error: Expected combined redirection '2>&1'.\n");
    return 0;
}

/*
    Separate command from redirection.
*/

*redirect = '\0';

strcpy(command, input);

/*
    Everything between > and 2>&1
    is the filename.
*/

redirect++;

char *filename_end = strstr(redirect, "2>&1");

if (filename_end == NULL)
{
    redirect++;

    char *filename_end = strstr(redirect, "2>&1");

    if (filename_end == NULL)
    {
        return 0;
    }

    *filename_end = '\0';

    strcpy(filename, redirect);

    trim(command);
    trim(filename);

    if (strlen(command) == 0)
    {
        printf("Error: Command is empty.\n");
        return 0;
    }

    if (strlen(filename) == 0)
    {
        printf("Error: Output filename is empty.\n");
        return 0;
    }

    return 1;
}
```

```
);

if (fd == -1)
{
    perror("open");
    return;
}

printf("Output file opened: %s\n", filename);
printf("File descriptor: %d\n", fd);

/*
    Create child process.
*/

pid_t pid = fork();

if (pid < 0)
{
    perror("fork");
    close(fd);
    return;
}

if (pid == 0)
{
    /*

close(fd);

/*
    Parse command arguments.
*/

char command_copy[MAX_INPUT];
strcpy(command_copy, command);

char *args[MAX_ARGS];

int count = 0;

char *token = strtok(command_copy, " ");

while (token != NULL && count < MAX_ARGS - 1)
{
    args[count++] = token;

    token = strtok(NULL, " ");
}

args[count] = NULL;

/*
    Execute command.
*/
```

```

}

input[strcspn(input, "\n")] = '\0';

/* Parse combined redirection.
*/

if (!parse_combined(
    input,
    command,
    filename))
{
    printf("\nInvalid combined redirection.\n");
    return 1;
}

printf("\nParsed Command: %s\n", command);
printf("Output File: %s\n", filename);

printf("\nExecuting combined redirection...\n");

execute_combined(command, filename);

printf("\n==== Combined Redirection Test Completed =====\n");

return 0;
}

```

To Parse Complex Commands, Combine Pipes and Redirections, Resolve Execution Order, Validate Syntax, Generate Execution Plans, Test Scenarios

```

divya@Divyasree:~$ cd
divya@Divyasree:~$ mkdir complex_command_project
divya@Divyasree:~$ cd complex_command_project
divya@Divyasree:~/complex_command_project$ pwd
/home/divya/complex_command_project
divya@Divyasree:~/complex_command_project$ nano input.txt
divya@Divyasree:~/complex_command_project$ cat input.txt
Linux File I/O Management
Linux processes and pipes
File descriptors in Linux
Ubuntu operating system
Data communication using pipes
Linux system calls
divya@Divyasree:~/complex_command_project$ nano complex_command.c
divya@Divyasree:~/complex_command_project$ gcc complex_command.c -o complex_command
divya@Divyasree:~/complex_command_project$ ls
complex_command  complex_command.c  input.txt
divya@Divyasree:~/complex_command_project$ ./complex_command
===== Complex Command Manager =====

Supported features:
| Pipeline
< Input redirection
> Output redirection
>> Append redirection
2> Error redirection
2>&1 Merge stderr with stdout

Example:
cat input.txt | grep Linux > result.txt 2>&1

Enter command:
> cat input.txt | grep Linux > result.txt 2>&1

```

===== EXECUTION PLAN =====

Stage 1:

Command: cat input.txt
Output: PIPE 1

Stage 2:

Command: grep Linux
Input : PIPE 1
Output: result.txt
Error : MERGED WITH STDOUT

=====

Execution plan is VALID.

Executing plan...

All pipeline processes completed.

==== Complex Command Test Completed =====

divya@Divyasree:~/complex_command_project\$ cat result.txt

Linux File I/O Management

Linux processes and pipes

File descriptors in Linux

Linux system calls

divya@Divyasree:~/complex_command_project\$./complex_command

==== Complex Command Manager =====

Supported features:

| Pipeline
< Input redirection

Example:

cat input.txt | grep Linux > result.txt 2>&1

Enter command:

> cat < input.txt | grep Linux

===== EXECUTION PLAN =====

Stage 1:

Command: cat
Input : input.txt
Output: PIPE 1

Stage 2:

Command: grep Linux
Input : PIPE 1

=====

Execution plan is VALID.

Executing plan...

Linux File I/O Management

Linux processes and pipes

File descriptors in Linux

Linux system calls

All pipeline processes completed.

==== Complex Command Test Completed =====

divya@Divyasree:~/complex_command_project\$./complex_command

==== Complex Command Manager =====

```
=====
Execution plan is VALID.
Executing plan...
All pipeline processes completed.

==== Complex Command Test Completed ====
divya@Divyasree:~/complex_command_project$
divya@Divyasree:~/complex_command_project$
divya@Divyasree:~/complex_command_project$
divya@Divyasree:~/complex_command_project$ ./complex_command
==== Complex Command Manager ====

Supported features:
| Pipeline
< Input redirection
> Output redirection
>> Append redirection
2> Error redirection
2>&1 Merge stderr with stdout

Example:
cat input.txt | grep Linux > result.txt 2>&1

Enter command:
> ls /does_not_exist > combined.txt 2>&1

===== EXECUTION PLAN =====
```

Stage 1:

```
divya@Divyasree:~/complex_command_project$ cat combined.txt
ls: cannot access '/does_not_exist': No such file or directory
divya@Divyasree:~/complex_command_project$ ./complex_command
==== Complex Command Manager =====
```

```
Supported features:
| Pipeline
< Input redirection
> Output redirection
>> Append redirection
2> Error redirection
2>&1 Merge stderr with stdout
```

Example:
cat input.txt | grep Linux > result.txt 2>&1

Enter command:
> cat input.txt | | grep Linux
Syntax error: empty command.

Syntax validation failed.
divya@Divyasree:~/complex_command_project\$./complex_command
==== Complex Command Manager =====

```
Supported features:
| Pipeline
< Input redirection
> Output redirection
>> Append redirection
2> Error redirection
2>&1 Merge stderr with stdout
```

Example:

```

2>&1 Merge stderr with stdout

Example:
cat input.txt | grep Linux > result.txt 2>&1

Enter command:
> cat < missing.txt | grep Linux

===== EXECUTION PLAN =====

Stage 1:
  Command: cat
  Input : missing.txt
  Output: PIPE 1

Stage 2:
  Command: grep Linux
  Input : PIPE 1

=====

Execution plan is VALID.

Executing plan...
Input file: No such file or directory

All pipeline processes completed.

===== Complex Command Test Completed =====
divya@Divyasree:~/complex_command_project$ |

```

To Detect Background Jobs, Launch Non-Blocking Processes, Return Prompt Immediately, Store Job Information, Monitor Execution, Test Background Tasks

```

divya@Divyasree:~$ cd
divya@Divyasree:~$ mkdir background_job_project
divya@Divyasree:~$ cd background_job_project
divya@Divyasree:~/background_job_project$ pwd
/home/divya/background_job_project
divya@Divyasree:~/background_job_project$ nano background_jobs.c
divya@Divyasree:~/background_job_project$ gcc background_jobs.c -o background_jobs
divya@Divyasree:~/background_job_project$ ls
background_jobs  background_jobs.c
divya@Divyasree:~/background_job_project$ ./background_jobs
===== Background Job Manager =====

Commands:
  command      - foreground command
  command &    - background command
  jobs         - display jobs
  exit         - exit program

mysh> sleep 10 &
Background job detected.
[1] 2714 running sleep 10

mysh> jobs

[1] Done: sleep 10

===== Background Jobs =====
[1] PID=2714  DONE  sleep 10
=====

mysh> jobs

===== Background Jobs =====

```

```

===== Background Jobs =====
[1] PID=2714  DONE  sleep 10
=====

mysh> sleep 5 &
Background job detected.
[2] 2982 running sleep 5

mysh> sleep 8 &
Background job detected.
[3] 3050 running sleep 8

[2] Done: sleep 5

mysh> jobs

[3] Done: sleep 8

===== Background Jobs =====
[1] PID=2714  DONE  sleep 10
[2] PID=2982  DONE  sleep 5
[3] PID=3050  DONE  sleep 8
=====

mysh> pwd
/home/divya/background_job_project

mysh> ls
background_jobs  background_jobs.c

mysh> echo Hello &
Background job detected.
[4] 3401 running echo Hello
^C
=====

mysh> pwd
/home/divya/background_job_project

mysh> ls
background_jobs  background_jobs.c

mysh> echo Hello &
Background job detected.
[4] 3401 running echo Hello

mysh> Hello
^C
divya@Divyasree:~/background_job_project$ |

```

Code :

```
GNU nano 7.2                                     backgrou
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <ctype.h>

#define MAX_JOBS 20
#define MAX_COMMAND 200

typedef struct
{
    int job_id;
    pid_t pid;
    char command[MAX_COMMAND];
    int running;
} Job;

Job jobs[MAX_JOBS];

int job_count = 0;
int next_job_id = 1;

/*
    Remove newline and trailing spaces.
*/
void clean_input(char *input)
{

```

```
    int len = strlen(input);

    while (len > 0 && isspace(input[len - 1]))
    {
        input[len - 1] = '\0';
        len--;
    }
}

/*
Detect whether command ends with &.
*/
int is_background(char *input)
{
    int len = strlen(input);

    while (len > 0 && isspace(input[len - 1]))
    {
        len--;
    }

    if (len > 0 && input[len - 1] == '&')
    {
        input[len - 1] = '\0';

        while (len > 1 && isspace(input[len - 2]))
        {
```

```
    if (len > 0 && input[len - 1] == '&')
    {
        input[len - 1] = '\\0';

        while (len > 1 && isspace(input[len - 2]))
        {
            input[len - 2] = '\\0';
            len--;
        }

        return 1;
    }

    return 0;
}
```

```
/*
    Store background job information.
*/
void store_job(pid_t pid, const char *command)
{
    if (job_count >= MAX_JOBS)
    {
        printf("Job table is full.\\n");
        return;
    }

    jobs[job_count].job_id = next_job_id++;
```

```
    jobs[job_count].job_id = next_job_id++;
    jobs[job_count].pid = pid;

    strncpy(
        jobs[job_count].command,
        command,
        MAX_COMMAND - 1
    );

    jobs[job_count].command[MAX_COMMAND - 1] = '\\0';

    jobs[job_count].running = 1;

    printf("[%d] %d running %s\\n",
        jobs[job_count].job_id,
        jobs[job_count].pid,
        jobs[job_count].command);

    job_count++;
}

/*
    Monitor background jobs.

    WNOHANG means:
    do not wait if the child is still running.
*/
void monitor_jobs()
```

```
void monitor_jobs()
{
    for (int i = 0; i < job_count; i++)
    {
        if (!jobs[i].running)
            continue;

        int status;

        pid_t result = waitpid(
            jobs[i].pid,
            &status,
            WNOHANG
        );

        if (result == 0)
        {
            /*      Child is still running.
            */
            continue;
        }

        if (result == jobs[i].pid)
        {
            jobs[i].running = 0;

            if (WIFEXITED(status))
            {
```

```
        if (WIFEXITED(status))
        {
            printf("\n[%d] Done: %s\n",
                    jobs[i].job_id,
                    jobs[i].command);
        }
        else
        {
            printf("\n[%d] Process terminated.\n",
                    jobs[i].job_id);
        }
    }
}
```

```
/*
    Display background jobs.
*/
```

```
void display_jobs()
{
    monitor_jobs();

    printf("\n==== Background Jobs =====\n");

    if (job_count == 0)
    {
        printf("No background jobs.\n");
        printf("===== \n");
    }
}
```

```
if (job_count == 0)
{
    printf("No background jobs.\n");
    printf("===== \n");
    return;
}

for (int i = 0; i < job_count; i++)
{
    printf("[%d] PID=%d %s %s\n",
            jobs[i].job_id,
            jobs[i].pid,
            jobs[i].running ? "RUNNING" : "DONE",
            jobs[i].command);
}

printf("===== \n");
}
```

```
/*
    Execute a background command.
*/
```

```
void execute_background(char *command)
{
    pid_t pid = fork();

    if (pid < 0)
    {

```

```

    }

    if (pid == 0)
    {
        /*
         * Child process.
         */

        char command_copy[MAX_COMMAND];

        strcpy(command_copy, command);

        char *args[30];

        int count = 0;

        char *token = strtok(
            command_copy,
            " ");

        while (token != NULL && count < 29)
        {
            args[count++] = token;

            token = strtok(NULL, " ");
        }

        args[count] = NULL;

    }

    if (pid == 0)
    {
        char command_copy[MAX_COMMAND];

        strcpy(command_copy, command);

        char *args[30];

        int count = 0;

        char *token = strtok(
            command_copy,
            " ");

        while (token != NULL && count < 29)
        {
            args[count++] = token;

            token = strtok(NULL, " ");
        }

        args[count] = NULL;

        execvp(args[0], args);

        perror("execvp");
    }

```

```
{
    char input[MAX_COMMAND];

    printf("==== Background Job Manager ====\\n");

    printf("\\nCommands:\\n");
    printf("  command      - foreground command\\n");
    printf("  command &    - background command\\n");
    printf("  jobs         - display jobs\\n");
    printf("  exit         - exit program\\n");

    while (1)
    {
        /*
         * Check whether any background
         * process has finished.
         */
        monitor_jobs();

        printf("\\nmysh> ");
        fflush(stdout);

        if (fgets(
            input,
            sizeof(input),
            stdin) == NULL)
        {
            break;
        }
        printf("Exiting Background Job Manager...\\n");

        /*
         * Check final job status.
         */
        monitor_jobs();

        break;
    }

    /*
     * Display jobs.
     */
    if (strcmp(input, "jobs") == 0)
    {
        display_jobs();
        continue;
    }

    /*
     * Detect &.
     */
    int background = is_background(input);

    if (strlen(input) == 0)
    {
        printf("Error: Empty command.\\n");
        continue;
    }
}
```

```

        if (strlen(input) == 0)
        {
            printf("Error: Empty command.\n");
            continue;
        }

        if (background)
        {
            printf("Background job detected.\n");

            execute_background(input);

            /*
             IMPORTANT:
             No waitpid() here.

             Prompt returns immediately.
            */
        }
        else
        {
            execute_foreground(input);
        }
    }

    return 0;
}

```

To Create Job Table, Store Process Groups, Track States, Update Status, Remove Completed Jobs, Validate Job Records

```

divya@Divyasree:~$ cd
divya@Divyasree:~$ mkdir job_table_project
divya@Divyasree:~$ cd job_table_project
divya@Divyasree:~/job_table_project$ pwd
/home/divya/job_table_project
divya@Divyasree:~/job_table_project$ nano job_table.c
divya@Divyasree:~/job_table_project$ gcc job_table.c -o job_table
divya@Divyasree:~/job_table_project$ ls
job_table  job_table.c
divya@Divyasree:~/job_table_project$ ./job_table
===== Job Table Manager =====

Commands:
command & - create background job
jobs       - display job table
cleanup    - remove completed jobs
validate   - validate job records
exit       - exit program

jobsh> sleep 10 &
Background job detected.
[1] PID=10163 PGID=10163 RUNNING sleep 10

jobsh> jobs

[1] Job completed: sleep 10

===== JOB TABLE =====
[1] PID=10163 PGID=10163 DONE      sleep 10
=====

jobsh> validate

```

```
==== Job Record Validation ====  
[1] VALID - PID=10163 PGID=10163 STATE=DONE  
=====
```

```
jobsh> jobs
```

```
===== JOB TABLE =====  
[1] PID=10163 PGID=10163 DONE      sleep 10  
=====
```

```
jobsh> cleanup  
Removing completed job [1]: sleep 10  
Job table cleanup completed.
```

```
jobsh> jobs
```

```
===== JOB TABLE =====  
Job table is empty.  
=====
```

```
jobsh> sleep 5 &  
Background job detected.  
[2] PID=12296 PGID=12296 RUNNING sleep 5
```

```
jobsh> sleep 10 &  
Background job detected.  
[3] PID=12332 PGID=12332 RUNNING sleep 10
```

```
[2] Job completed: sleep 5
```

```
jobsh> sleep 15 &  
Background job detected.
```

```

===== JOB TABLE =====
[2] PID=12296 PGID=12296 DONE      sleep 5
[3] PID=12332 PGID=12332 DONE      sleep 10
[4] PID=12475 PGID=12475 RUNNING  sleep 15
=====

jobsh> validate

[4] Job completed: sleep 15

===== Job Record Validation =====
[2] VALID - PID=12296 PGID=12296 STATE=DONE
[3] VALID - PID=12332 PGID=12332 STATE=DONE
[4] VALID - PID=12475 PGID=12475 STATE=DONE
=====

jobsh> jobs

===== JOB TABLE =====
[2] PID=12296 PGID=12296 DONE      sleep 5
[3] PID=12332 PGID=12332 DONE      sleep 10
[4] PID=12475 PGID=12475 DONE      sleep 15
=====

jobsh> cleanup
Removing completed job [2]: sleep 5
Removing completed job [3]: sleep 10
Removing completed job [4]: sleep 15
Job table cleanup completed.

jobsh> jobs

```

```

===== JOB TABLE =====
Job table is empty.
=====

jobsh> ^C
divya@Divyasree:~/job_table_project$ |

```

Code:

```

GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <ctype.h>

#define MAX_JOBS 20
#define MAX_COMMAND 200

typedef enum
{
    JOB_RUNNING,
    JOB_STOPPED,
    JOB_DONE
} JobState;

typedef struct
{
    int job_id;
    pid_t pid;
    pid_t pgid;
    char command[MAX_COMMAND];
    JobState state;
} Job;

Job job_table[MAX_JOBS];

int job_count = 0;
int next_job_id = 1;

/* Convert state to readable text */
const char *state_name(JobState state)
{
    switch (state)
    {
        case JOB_RUNNING:
            return "RUNNING";

        case JOB_STOPPED:
            return "STOPPED";

        case JOB_DONE:
            return "DONE";

        default:
            return "UNKNOWN";
    }
}

/* Remove newline and extra spaces */

```

```
GNU nano 7.2
*/
int validate_job(Job *job)
{
    if (job->job_id <= 0)
        return 0;

    if (job->pid <= 0)
        return 0;

    if (job->pgid <= 0)
        return 0;

    if (strlen(job->command) == 0)
        return 0;

    if (job->state < JOB_RUNNING ||
        job->state > JOB_DONE)
    {
        return 0;
    }

    return 1;
}

/* Add a new job to the job table.
*/
int add_job(pid_t pid,
            pid_t pgid,
            const char *command)
{
    if (job_count >= MAX_JOBS)
    {
        printf("Job table is full.\n");
        return -1;
    }

    Job *job = &job_table[job_count];

    job->job_id = next_job_id++;

    job->pid = pid;

    job->pgid = pgid;

    strncpy(
        job->command,
        command,
        MAX_COMMAND - 1
    );

    job->command[MAX_COMMAND - 1] = '\0';
}
```

```

/* Update the state of a job.
*/
void update_job_status()
{
    for (int i = 0; i < job_count; i++)
    {
        Job *job = &job_table[i];

        if (job->state == JOB_DONE)
            continue;

        int status;

        pid_t result = waitpid(
            job->pid,
            &status,
            WNOHANG | WUNTRACED
        );

        if (result == 0)
        {
            /* Child is still running.
            */

            job->state = JOB_RUNNING;
        }
        else if (result == job->pid)
        {
            if (WIFSTOPPED(status))
            {
                job->state = JOB_STOPPED;

                printf(
                    "\n[%d] Job stopped: %s\n",
                    job->job_id,
                    job->command
                );
            }
            else if (WIFEXITED(status) ||
                WIFSIGNALED(status))
            {
                job->state = JOB_DONE;

                printf(
                    "\n[%d] Job completed: %s\n",
                    job->job_id,
                    job->command
                );
            }
        }
    }
}

```

```

GNU nano 7.2
void display_jobs()
{
    update_job_status();

    printf("\n===== JOB TABLE =====\n");

    if (job_count == 0)
    {
        printf("Job table is empty.\n");
        printf("=====\n");
        return;
    }

    for (int i = 0; i < job_count; i++)
    {
        Job *job = &job_table[i];

        printf(
            "[%d] PID=%d PGID=%d %-8s %s\n",
            job->job_id,
            job->pid,
            job->pgid,
            state_name(job->state),
            job->command
        );
    }

    printf("=====\n");
}

/* Remove completed jobs from the table.
*/
void remove_completed_jobs()
{
    int new_count = 0;

    for (int i = 0; i < job_count; i++)
    {
        if (job_table[i].state == JOB_DONE)
        {
            printf(
                "Removing completed job [%d]: %s\n",
                job_table[i].job_id,
                job_table[i].command
            );

            continue;
        }

        /* Keep unfinished job.

```

```

        job_table[new_count] =
            job_table[i];
    }

    new_count++;
}

job_count = new_count;
}

/*
 * Execute a background job.
 */
void launch_background(char *command)
{
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return;
    }

    if (pid == 0)
    {
        /*
         * Child becomes leader of its own
         * process group.
         */

        if (setpgid(0, 0) == -1)
        {
            perror("setpgid");
            exit(1);
        }

        char command_copy[MAX_COMMAND];

        strcpy(command_copy, command);

        char *args[30];

        int count = 0;

        char *token =
            strtok(command_copy, " ");

        while (token != NULL &&
            count < 29)
        {
            args[count++] = token;

            token = strtok(NULL, " ");

```

```

char input[MAX_COMMAND];

printf("==== Job Table Manager =====\n");

printf("\nCommands:\n");
printf("  command & - create background job\n");
printf("  jobs      - display job table\n");
printf("  cleanup   - remove completed jobs\n");
printf("  validate  - validate job records\n");
printf("  exit      - exit program\n");

while (1)
{
    /*
     * Update states whenever prompt appears.
     */

    update_job_status();

    printf("\njobsh> ");

    fflush(stdout);

    if (fgets(
        input,
        sizeof(input),
        stdin) == NULL)
    {
        break;
    }

    clean_input(input);

    if (strlen(input) == 0)
        continue;

    /*
     * Exit.
     */

    if (strcmp(input, "exit") == 0)
    {
        printf(
            "Exiting Job Table Manager...\n"
        );

        break;
    }

    /*
     * Display job table.
     */
}

```

```

        update_job_status();

        remove_completed_jobs();

        printf(
            "Job table cleanup completed.\n"
        );

        continue;
    }

    /*
     * Validate all job records.
     */

    if (strcmp(input, "validate") == 0)
    {
        update_job_status();

        printf(
            "\n==== Job Record Validation =====\n"
        );

        if (job_count == 0)
        {
            printf("No job records to validate.\n");
        }

        for (int i = 0;
             i < job_count;
             i++)
        {
            if (validate_job(
                &job_table[i]))
            {
                printf(
                    "[%d] VALID - PID=%d PGID=%d STATE=%s\n",
                    job_table[i].job_id,
                    job_table[i].pid,
                    job_table[i].pgid,
                    state_name(
                        job_table[i].state
                    )
                );
            }
            else
            {
                printf(
                    "[%d] INVALID JOB RECORD\n",
                    job_table[i].job_id
                );
            }
        }
    }
}

```

```

    );
    continue;
}

/*
Detect background '&'.
*/
int len = strlen(input);
if (len > 0 &&
    input[len - 1] == '&')
{
    input[len - 1] = '\0';
    clean_input(input);

    if (strlen(input) == 0)
    {
        printf(
            "Error: Empty background command.\n"
        );
        continue;
    }

    printf(
        "Background job detected.\n"
    );
    launch_background(input);

    /*
    Parent immediately returns
    to the prompt.
    */
    continue;
}

/*
Commands without & are not supported
as foreground jobs in this exercise.
*/
printf(
    "Use '&' to create a background job.\n"
);
}

return 0;
}
|

```